

Howard University
College of Engineering & Architecture
Department of Electrical Engineering & Computer Science
EECE 306-01: Fundamentals of Electromagnetics Laboratory
Dr. Su Yan, su.yan@howard.edu

Lab 3: Point Charges, Superposition, and the Field Engine

Objectives

By the end of this session you will have

- built the source representation that the entire rest of the toolbox depends on,
- implemented the electric field evaluator and made it fast enough for grading,
- written your first symmetry test, which needs no reference answer,
- produced a field plot that is actually readable.

1. One decision, ten weeks of consequences

Today you make the single most consequential design choice in the toolbox, so it is worth understanding before you type anything.

A source in this toolbox is not a formula. It is a struct holding a **discretized list of differential elements**.

```
s.type % 'charge' or 'current'
s.pos % Mx3, element positions
s.w % Mx1, quadrature weights, units m, m^2, or m^3
s.q % Mx1, charge density at each element, when type is 'charge'
s.Idl % Mx3, the vector I dL at each element, when type is 'current'
s.label % char, optional
```

A point charge is the case $M = 1$, with $w = 1$ and q holding the total charge in coulombs. That looks like a strange special case until you notice that it makes the product $q \cdot w$ carry units of coulombs in *every* case, whether the source is a point charge, a line charge with w in meters, a surface charge with w in square meters, or a volume charge with w in cubic meters. One evaluator, consistent units, no special-casing.

The payoff arrives in two instalments. Next week you will write three continuous charge constructors and `em.field.E` will need no modification to handle any of them. In Lab 9 you will write the magnetic field evaluator and discover that it is the same loop with a different numerator, and that the quadrature machinery you build next week transfers to magnetostatics without a single change.

The general principle

The alternative design, a separate field formula for each source geometry, is what most textbooks present and it is what most first attempts look like. It works, and it produces a codebase where adding a new source shape means touching every field routine. Separating *what the source is* from *how the field is computed* costs a little thought today and saves the rest of the semester. Notice which parts of your code know about geometry and which parts know about physics, and keep them apart.

2. Part I. The source constructors

2.1 `em.src.pointCharge(Q, r0)`

Returns a struct with `type` set to `'charge'`, `pos` equal to the 1x3 position, `w` equal to 1, and `q` equal to Q in coulombs.

Validate the inputs. A position that is not 1x3 must raise an error naming the argument. Silent acceptance of a column vector is a bug that will cost somebody an hour in Lab 5.

2.2 `em.src.merge(s1, s2, ...)`

Concatenates the element lists of any number of sources of the same type. This is superposition, made into an explicit operation you can see in the code.

It must `error()` when asked to merge a `'charge'` source with a `'current'` source. Those two are added by superposing their *fields*, never their element lists, and a toolbox that silently allows the mistake will produce a plausible looking number that is meaningless.

3. Part II. The field engine

Implement `em.field.E(s, r)`. Input is a source struct and an Nx3 array of observation points. Output is Nx3 in V/m.

$$\vec{E}(\vec{r}) = \frac{1}{4\pi\epsilon_0} \sum_{i=1}^M q_i w_i \frac{\vec{r} - \vec{r}_i}{|\vec{r} - \vec{r}_i|^3}$$

Three requirements beyond the formula.

Vectorize. From Lab 5 onward this engine is called with up to 10^4 observation points against sources with up to 10^4 elements. A double loop over both will not finish. Build the $N \times M$ displacement array with implicit expansion, or chunk the observation points if memory becomes the binding constraint. Report in your notebook the largest $N \times M$ product your machine handles before it slows down, and say what limited it.

Guard the singularity. The field at the location of a point charge is infinite. When an observation point falls within a tolerance of an element position, raise an error that names the offending element index. Returning `Inf` silently is worse, because it propagates into a plot and produces a picture that looks merely odd rather than wrong.

Do not hardcode ϵ_0 . Call `em.const.eps0()`. That is what Lab 1 was for.

4. Part III. Three tests, three different kinds

4.1 Test A, against a closed form

A single point charge Q at the origin produces

$$\vec{E} = \frac{Q}{4\pi\epsilon_0 r^2} \hat{r}$$

Compare your evaluator against this at a spread of radii and directions, with a *relative* tolerance. This is the familiar kind of test and it catches gross errors.

4.2 Test B, by symmetry, and this is the interesting one

Place two *equal* charges Q at $(-d, 0, 0)$ and $(+d, 0, 0)$. On the plane $x = 0$, the perpendicular bisector, the x component of the total field must vanish exactly, by symmetry. There is no cancellation to be arranged and no formula to look up. The physics guarantees it, so your code must reproduce it.

```
q1 = em.src.pointCharge(1e-9, [-0.1 0 0]);
q2 = em.src.pointCharge(1e-9, [ 0.1 0 0]);
s = em.src.merge(q1, q2);
p = [zeros(200,1), randn(200,1), randn(200,1)]; % the x = 0 plane
Ev = em.field.E(s, p);
tol = 1e-6 * max(em.vec.mag(Ev)); % tiny compared with the field itself
em.test.assertClose(max(abs(Ev(:,1))), 0, tol, 'bisector symmetry');
```

Why the tolerance is absolute here

The expected value is zero, so a relative comparison would divide by zero. This is the case `em.test.assertClose` handles with its absolute branch. Set the threshold to a small fraction of the field magnitude at that point, as shown above, rather than to a fixed number of volts per meter, so the same test works whatever the charge and distance happen to be.

4.3 Test C, by scaling

Place two *opposite* charges a small distance apart to form a dipole. Far from the pair the field magnitude must fall as $1/r^3$, not $1/r^2$, because the leading monopole term cancels. Evaluate the field magnitude at radii r and $2r$ with both large compared with the separation, and check that the ratio approaches 8.

This is a third kind of test again. No closed form is invoked and no exact symmetry, only the expected scaling of a limit. It is the same reasoning you will use in Lab 4 to decide when a finite line charge starts to look infinite.

5. Part IV. Making the picture readable

Implement `em.viz.quiver2(F, xlim, ylim, N, opts)`. It takes a field *function handle*, samples it on a grid in a plane, and draws arrows.

Plot the field of a single point charge with `opts.normalize` false. The arrows near the charge are so long that everything else is invisible, because the magnitude spans several orders of magnitude across the plot. Now set `opts.normalize` true, so that every arrow has the same length and only the direction is shown. The structure of the field becomes visible immediately.

Both plots belong in your notebook, side by side, with one sentence on what each is good for. A magnitude plot answers how strong, a direction plot answers which way, and trying to make one picture answer both usually answers neither.

Required tests

Write `tests/test_lab03.m` containing at least these checks.

1. Single point charge against the closed form at ten radii, relative tolerance 10^{-12} .
2. Field direction is radially outward for positive Q and inward for negative Q .
3. Two equal charges, transverse component vanishes on the bisector plane, absolute tolerance

justified in your notebook.

4. Dipole far field ratio approaches 8 between r and $2r$, within five percent.
5. `em.src.merge` of two charge sources produces an element count equal to the sum of the two.
6. `em.src.merge` of a charge source with a current source raises an error. Use `try` and `catch`.
7. Evaluating the field at the exact location of a point charge raises an error.
8. `em.field.E` returns $N \times 3$ for an $N \times 3$ input with $N = 5000$, and completes in under two seconds.

Deliverables

Submit through Canvas by the start of the next lab session.

1. **Toolbox snapshot**, `TeamNN_Lab03.zip`.
2. **Lab notebook**, published script exported to PDF, three to five pages, containing the two quiver plots, your tolerance justification for Test B, and your timing report.
3. **Test output** pasted into the notebook.

Looking ahead

Next week is the heaviest build of the semester and the one everything else rests on, the quadrature engine. Once it exists, line charges, surface charges, volume charges, closed path integrals and closed surface integrals all become one line of code each. Read Sections 2.4 and 2.5 of Wentworth before you come.